

Table of Contents

AlphaX POS — Implementer & Developer Manual

For: Frappe developers, partners, and technical implementers **Version:** 15.5.13 **Last updated:** May 2026

What's in this manual

1. [Overview for the developer](#)
 2. [Repository structure and layout](#)
 3. [The three-app architecture in detail](#)
 4. [Frappe Cloud deployment specifics](#)
 5. [The cashier UI: classic vs Vue, runtime SFC loader](#)
 6. [Doctype reference: all 52, what they do](#)
 7. [Hooks and triggers reference](#)
 8. [Boot API contract](#)
 9. [Manager PIN security module](#)
 10. [Domain pack system](#)
 11. [Soft adapter pattern \(ZATCA, hardware bridge\)](#)
 12. [Custom field strategy: no migrations](#)
 13. [Extending the system](#)
 14. [Customer onboarding playbook](#)
 15. [Known issues and gotchas](#)
 16. [Roadmap and deferred work](#)
-

Overview for the developer

✅ **Most of this manual is verified working.** Sections marked 🧐 or ⚠️ follow the same conventions as other manuals.

You're reading this because you're either:

- A Frappe developer asked to extend AlphaX POS
- A Frappe partner deploying it for a customer
- The maintainer of the codebase reviewing the architecture

The codebase is the result of significant design work. Everything here is documented for one purpose: **so you don't have to read the entire codebase to be productive in 24 hours.**

What you should already know

- Frappe v15 framework: doctypes, hooks, REST API, fixtures, scheduler
- ERPNext: at least the POS, Sales Invoice, Stock, Item modules
- Python 3.10+, JavaScript ES2017+, jQuery (sadly), Vue 3 (optional)
- Frappe Cloud's deploy model (GitHub-based)

What you don't need to know

- ZATCA's e-invoicing standards in detail (the soft adapter shields you from most of it)
 - Bcrypt internals (passlib does the work)
 - USB hardware protocols (the bridge daemon handles it)
-

Repository structure and layout

✓ Working

The main repo is on GitHub:

<https://github.com/jamunachi08/alphax-pos-core-v15.5.1>

Local checkout layout:

```
alphax-pos-core/                                <-- repo root
├── setup.py                                    (Frappe app metadata)
├── pyproject.toml
├── MANIFEST.in
├── README.md
├── license.txt                                (Proprietary)
├── build_workspace.py                         (utility to regenerate
workspace fixture)                           (one-time: downloads
├── fetch_vendor_bundles.py
Vue/Pinia/vue-il8n)
├── docs/                                     (this manual + the other 3)
│   ├── 01-cashier-manual.md
│   ├── 02-manager-manual.md
│   ├── 03-administrator-manual.md
│   ├── 04-implementer-manual.md
│   └── exports/                             (generated .docx and .pdf)
├── alphax_pos_suite/                         (the app package)
│   ├── __init__.py                          (__version__ string)
│   ├── hooks.py                             (ALL hook registrations)
│   ├── modules.txt                          ("AlphaX POS Suite")
│   ├── patches.txt                          (migration patches)
│   ├── fixtures/
│   │   └── workspace.json                   (the AlphaX POS Hub)
│   └── frontend/                           (Vite source – DO NOT BUILD;
reference only)
```

```

├── DO_NOT_BUILD.md
├── alphas_pos_suite/           (the module folder)
│   ├── doctype/               (~52 doctypes)
│   ├── page/
│   │   ├── alphas_pos_classic/ (the working cashier – jQuery +
│   │   │   └── alphas_pos_v2/   (Vue cashier – currently 🐛
│   │   │   ├── alphas_apos/
│   │   │   ├── alphas_floor_designer/
│   │   │   ├── alphas_kds/
│   │   │   ├── alphas_pos_central_kitchen_dashboard/
│   │   │   ├── alphas_pos_profitability/
│   │   │   └── alphas_pos_setup/
│   │   └── public/
│   │       ├── dist/
│   │       │   └── vendor/      (assets that work – the only
│   │       │   ├── css/
│   │       │   │   ├── alphas_pos_hub.css
│   │       │   │   └── alphas_pos_classic.css
│   │       │   ├── js/
│   │       │   │   ├── sales_invoice_terminal_capture.js
│   │       │   │   ├── bonanza_pos_warnings.js
│   │       │   │   └── alphas_workspace_theme_apply.js
│   │       │   ├── cashier/     (Vue SPA assets)
│   │       │   │   ├── main.js
│   │       │   │   ├── sfc-loader.js
│   │       │   │   └── sfc/     (.vue files)
│   │       │   ├── vue.global.prod.js
│   │       │   ├── pinia.iife.prod.js
│   │       │   └── vue-il8n.global.prod.js
│   │       ├── boot/
│   │       │   ├── api.py       (whitelisted boot endpoints)
│   │       │   ├── security/
│   │       │   │   └── manager_pin.py (PIN auth, logout, audit)
│   │       │   ├── appearance/
│   │       │   │   └── workspace_theme_css.py (boot_session hook for theme
│   │       │   ├── pos/         (sales, posting, processing
│   │       │   ├── integrations/ (zatca_adapter, card_capture,
│   │       │   ├── pharmacy/    (drug interaction checks,
│   │       │   ├── loyalty/     (points calculation, expiry)
│   │       │   └── data/        (custom_fields_seed.json, role
│   │       ├── patches/v15_0/   (migration patches)
│   │       └── install.py       (after_install chain)

```

Frappe UI) Beta) safe folder!) toggle) logic) payfort) controlled substance) permissions)

⚠ **Important:** the GitHub repo name has hyphens and a version suffix (alphax-pos-core-v15.5.1), but the app's installed name is `alphax_pos_suite` (underscores). This works because of Frappe's `app_name` resolution, but causes occasional bugs. Long-term TODO: rename the repo to `alphax_pos_suite` for cleanliness.

The three-app architecture in detail

App 1: `alphax_pos_suite`

The main app. Contains everything you can see in this manual: doctypes, cashier UIs, workspace, manager PIN system, hooks.

Versioning: `15.5.13` follows semver, with the major matching the Frappe major (`15` = Frappe `v15`).

App 2: `alphax_zatca`

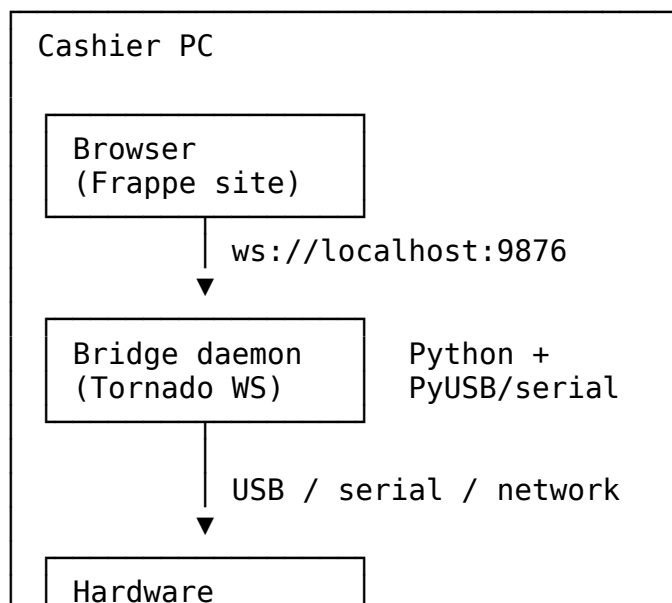
ZATCA Phase 2 e-invoicing. Forked from ERP Gulf's `zatca_erp_gulf`. Stays separate because:

- The fork retains GPL-3 license; we don't want it tainting the proprietary main app
- ZATCA changes (regulations, certificates) happen on a different cadence than the POS
- Customers outside KSA can install just the main app

The main app talks to ZATCA via a **soft adapter** (see “Soft adapter pattern” below) — the integration is loose enough that you can disable ZATCA entirely without breaking the POS.

App 3: `alphax-pos-bridge`

Standalone Python daemon. NOT a Frappe app. Runs on each cashier PC. Architecture:



- ESC/POS print
- Cash drawer
- Scale
- Card terminal
- Scanner (HID)

The bridge is **optional**. Without it, the cashier still works using the browser's native print dialog and HID barcode scanners. With it, auto-cut printing, drawer kicking, scale tare, and card-terminal data exchange become possible.

Communication contract

Bridge ↔ browser is JSON-over-WebSocket. Examples:

```
// Browser → Bridge
{ "id": "abc123", "action": "print_receipt",
  "args": { "html": "...", "open_drawer": true } }

// Bridge → Browser
{ "id": "abc123", "ok": true }
{ "event": "scale_reading", "weight_grams": 1247 }
```

The bridge is fully documented in its own repo's README.

Frappe Cloud deployment specifics

✓ **Working** with workarounds documented below.

The asset path workaround (v15.5.9)

Problem: On Frappe Cloud, the standard Frappe asset symlinking sometimes fails to symlink ALL of apps/alphax_pos_suite/alphax_pos_suite/public/ into sites/assets/alphax_pos_suite/. Files in dist/vendor/ get symlinked, but files in js/, css/, and dist/cashier/ do not.

Workaround: All static assets are colocated under dist/vendor/:

```
public/dist/vendor/
├── _css/           ← was public/css/
├── _js/           ← was public/js/
├── cashier/       ← was public/dist/cashier/
└── *.global.prod.js ← original vendor bundles
```

hooks.py uses these new paths:

```
app_include_css = [
    "/assets/alphax_pos_suite/dist/vendor/_css/alphax_pos_hub.css",
```

```
"/assets/alphax_pos_suite/dist/vendor/_css/alphax_pos_classic.css",
]
app_include_js = [
"/assets/alphax_pos_suite/dist/vendor/_js/sales_invoice_terminal_capture.js",
    ...
]
```

Why this works: the `dist/vendor/` folder happens to be the only one that survives the Frappe Cloud build pipeline reliably. Cause is unknown — possibly related to the build's hard-link strategy. We sidestep the issue rather than fight it.

If you find the root cause: great, but moving the folders back to their natural locations risks breaking deployments. Test exhaustively before reverting.

The vendor bundle commitment (v15.5.7)

We don't fetch Vue/Pinia/vue-il8n from CDN at runtime. They're committed to the repo at:

```
public/dist/vendor/vue.global.prod.js      (~160 KB)
public/dist/vendor/pinia.iife.prod.js      (~6 KB)
public/dist/vendor/vue-il8n.global.prod.js (~56 KB)
```

To upgrade these versions, edit `fetch_vendor_bundles.py`, run it on your laptop, commit the new files. Don't try to fetch them from within `bench install-app` — Frappe Cloud's deploy containers don't have outbound CDN access.

The deprecated package.json (v15.5.6)

A `package.json` at the top level of the app used to trigger a `cd frontend && npm install && npm run build` step during `bench build`. This always failed on Frappe Cloud and aborted the entire asset symlinking. We removed it.

Do not re-add a top-level package.json. If you need npm tooling for development, use `frontend/package.json` (which is gitignored from build pipelines).

The frappe.build.bundle fallback chain (v15.5.7)

In `install.py`, the `force_rebuild_assets()` helper tries three strategies:

1. `frappe.build.bundle()` (Python API, may not exist in all v15 minor versions)
2. `subprocess.run(['bench', 'build', '--app', 'alphax_pos_suite'])` (CLI)
3. Manual `os.symlink()` of `public/` to `sites/assets/`

Each fall-through is silent. If all three fail, the install proceeds anyway (assets remain unbuilt). The bench operator can then run `bench build` manually.

The cashier UI: classic vs Vue, runtime SFC loader

The two cashiers

Classic (/app/alphax-pos-classic) —  **Production**

- Plain Frappe page with jQuery + Frappe UI helpers
- HTML template inline in `alphax_pos_classic.js`
- Reactive only via manual DOM updates
- ~700 lines of JS, ~500 lines of CSS
- Loads in ~150ms, no vendor dependencies
- Recommended for v15.5.13 deployments

Vue (/app/alphax-pos-v2) —  **Beta**

- Vue 3 SPA with Pinia for state, `vue-i18n` for locales
- 30+ `.vue` files compiled at runtime by a custom SFC loader
- ~520 line SFC loader handles `<script setup>` syntax, scoped CSS, template compilation
- Modern reactive UI with kiosk mode, modifier dialogs, etc.
- Currently broken on `neol5` due to asset path issues; targeted for v15.6 production status

Both share the same backend (boot API, sales submission, etc.). You can run both in production — workspace shortcuts can point at either.

How the SFC loader works

The Vue cashier doesn't use Vite or any build step. Instead:

1. Page loads → loads `vue.global.prod.js`, `pinia`, `vue-i18n` from `/assets`
2. Page loads `sfc-loader.js` (the magic happens here)
3. Page loads `main.js` (mounts the Vue app)
4. `main.js` calls `sfc.loadComponent('App.vue')`
5. `sfc-loader` fetches `App.vue` as text from `/assets/.../sfc/App.vue`
6. `sfc-loader` splits `<template>`, `<script setup>`, `<style scoped>`
7. Compiles template with Vue's runtime template compiler
8. Transforms `<script setup>` to `setup()` (poor man's compiler)
9. Scopes CSS by adding `[data-v-XXX]` selectors
10. Returns a usable Vue component object
11. App mounts. Repeat for each child component on demand.

This avoids: - npm install during deployment - A build step before deployment - Source maps that leak proprietary code

It costs: - ~50ms longer initial load (template compile happens in browser) - Some Vue features (TypeScript) are unavailable - Source-map-based debugging is harder

The trade was deliberate. **Do not reintroduce a Vite build pipeline without serious reason.**

Boot sequence in detail

page/alphax_pos_v2/alphax_pos_v2.js:

```
async function bootCashier(wrapper) {
  // 1. Load vendor bundles, local first then CDN fallback
  await loadScript(VENDOR_LOCAL_BASE + '/vue.global.prod.js',
VENDOR_CDN_VUE);
  await loadScript(VENDOR_LOCAL_BASE + '/pinia.iife.prod.js',
VENDOR_CDN_PINIA);
  await loadScript(VENDOR_LOCAL_BASE + '/vue-i18n.global.prod.js',
VENDOR_CDN_I18N);

  // 2. Load global cashier styles (non-blocking <link>)
  loadStyle(SPA_LOCAL_BASE + '/sfc/styles/globals.css');

  // 3. Load SFC loader and main bootstrap
  await loadScript(SPA_LOCAL_BASE + '/sfc-loader.js');
  await loadScript(SPA_LOCAL_BASE + '/main.js');

  // 4. main.js exports `start(rootSelector, baseUrl)` which sfc-loads
  // App.vue and mounts it
  await window.AlphaXSPA.start('#alphax-cashier-app', SPA_LOCAL_BASE);
}
```

If any vendor file fails to load from BOTH local and CDN, a `VendorLoadError` is thrown and a “couldn’t reach display files” card is shown. Other failures show “Something went wrong loading the register”.

Doctype reference: all 52, what they do

✅ All listed doctypes exist in the codebase. Some are 🧪 Beta (noted).

Core POS

Doctype	Purpose
AlphaX POS Outlet	Business unit within a Branch
AlphaX POS Terminal	Physical cashier station
AlphaX POS Settings	Single doctype, global config
AlphaX POS Profile	Wraps standard POS Profile, adds AlphaX-specific fields
AlphaX POS Theme	Visual theme (colors, logos)
AlphaX POS Domain Pack	Feature flags per vertical

Orders & Sales

Doctype	Purpose
AlphaX POS Order	Held order (intermediate before Sales Invoice)
AlphaX POS Order Item	Child of POS Order
AlphaX POS Order Tax	Tax breakdown per order
AlphaX POS Order Payment	Payment line per order
AlphaX POS Invoice	(Currently unused; reserved for direct invoicing flow)

Shifts & Cash

Doctype	Purpose
AlphaX POS Shift	Cashier's open-to-close session
AlphaX POS Shift Cash Move	Cash drops, lifts, transfers
AlphaX POS Cash Move	Standalone cash movement
AlphaX POS Day Close	End-of-day summary

Returns & Credit Notes

Doctype	Purpose
AlphaX POS Return	Return record
AlphaX POS Return Line	Child of Return
AlphaX POS Credit Note	Customer credit balance
AlphaX POS Credit Note Use	Each redemption against a credit note

Loyalty & Offers

Doctype	Purpose
AlphaX POS Loyalty Program	Loyalty rules
AlphaX POS Loyalty Tier	Tier (Silver, Gold, etc.)
AlphaX POS Loyalty Customer Card	Customer's loyalty record
AlphaX POS Loyalty Transaction	Earn / redeem ledger
AlphaX POS Offer	Coupon code or promotion
AlphaX POS Offer Redemption	Each use of an offer

Pharmacy

Doctype	Purpose
AlphaX Drug Master	Drug catalog with schedule, max refills
AlphaX Prescription	Prescription record
AlphaX Prescription Line	Child of prescription
AlphaX Drug Interaction Rule	Pairs of dangerous drug combinations
AlphaX Controlled Substance Log	Audit log for Schedule II/III/IV dispenses

Doctype	Purpose
---------	---------

Restaurant

Doctype	Purpose
AlphaX POS Recipe	BOM-style item recipe
AlphaX POS Recipe Item	Ingredients child
AlphaX POS Modifier Group	Extras (size, milk, sauce)
AlphaX POS Modifier	Specific extra option
AlphaX POS Floor Plan	Restaurant tables layout
AlphaX POS Table	Individual table
AlphaX POS KDS Display	Kitchen Display configuration

Central Kitchen

Doctype	Purpose
AlphaX POS Central Kitchen Outlet	Outlet that feeds others
AlphaX POS Central Kitchen Map	Item routing rules

Compliance / ZATCA

Doctype	Purpose
AlphaX POS Processing Log	ZATCA submission status per invoice
AlphaX POS Audit Log	Generic audit (different from manager auth log)

Hardware integration

Doctype	Purpose
AlphaX POS Card Transaction	Card terminal capture record
AlphaX POS Receipt Template	Print template per profile
AlphaX POS Scale Profile	Weight scale config

Security (NEW in v15.5.13)

Doctype	Purpose
AlphaX POS Manager PIN	Bcrypt-hashed PIN per manager + lockout state
AlphaX POS Manager Authorization Log	Append-only audit trail

Configuration tables (Single doctypes)

Doctype	Purpose
AlphaX POS Settings	Global settings (44 fields)
AlphaX POS Hardware Settings	Bridge daemon connection params
AlphaX POS Loyalty Settings	Default loyalty config

💡 Use `bench --site <site> console` then `frappe.get_all("DocType", {"module": "AlphaX POS Suite"})` for the live list — it's the ground truth.

Hooks and triggers reference

Document hooks (`hooks.py::doc_events`)

```
"Sales Invoice": {
    "validate":
"alphax_pos_suite...integrations.card_capture.sales_invoice_validate",
    "before_submit":
"alphax_pos_suite...integrations.card_capture.sales_invoice_before_sub
mit",
    "on_submit": [

"alphax_pos_suite...integrations.card_capture.sales_invoice_on_submit"
,
        "alphax_pos_suite...pos.processing.on_sales_invoice_submit",
        "alphax_pos_suite...loyalty.hooks.on_sales_invoice_submit",

"alphax_pos_suite...integrations.zatca_adapter.on_pos_invoice_submit",
    ],
    "on_cancel": [
        "alphax_pos_suite...loyalty.hooks.on_sales_invoice_cancel",
    ],
},
"AlphaX POS Order": {
    "on_submit": "alphax_pos_suite...pos.posting.on_order_submit",
    "on_cancel": "alphax_pos_suite...pos.posting.on_order_cancel",
},
```

Lifecycle hooks

```
after_install =
"alphax_pos_suite.alphax_pos_suite.install.after_install"
after_migrate = [
    "alphax_pos_suite.alphax_pos_suite.install.force_rebuild_assets",

"alphax_pos_suite.alphax_pos_suite.install.fetch_vendor_bundles_silent
ly",
]
boot_session =
"alphax_pos_suite.alphax_pos_suite.appearance.workspace_theme_css.boot
_workspace_theme"
```

Scheduled tasks

```
scheduler_events = {
  "daily": [

    "alphax_pos_suite.alphax_pos_suite.pos.maintenance.daily_cleanup",

    "alphax_pos_suite.alphax_pos_suite.loyalty.hooks.expire_points",

    "alphax_pos_suite.alphax_pos_suite.security.manager_pin.reset_daily_counters",
  ],
}
```

after_install chain

In order:

1. `create_roles()` — creates AlphaX POS Manager, etc.
 2. `create_custom_fields()` — installs all from `data/custom_fields_seed.json`
 3. `create_workspace()` — loads fixtures/workspace.json
 4. `create_role_profiles()`
 5. `apply_permissions()`
 6. `seed_domain_packs()` — creates the 8 domain packs
 7. `fetch_vendor_bundles_silently()` — best-effort CDN fetch
 8. `force_rebuild_assets()` — three-strategy bench build fallback
-

Boot API contract

✓ Working

The `pos_boot` endpoint at `alphax_pos_suite.alphax_pos_suite.boot.api.pos_boot` is what the cashier UI calls on load to get its operational context.

Request

POST /api/method/alphax_pos_suite...boot.api.pos_boot
Content-Type: application/json

```
{ "terminal": "A07" }
```

Response shape

```
{
  "message": {
    "outlet": { "name": "Coffee Shop", "branch": "Riyadh Mall", ... },
    "profile": { ... pos profile fields ... },
    "domain_pack": { ... feature flags ... },
    "theme": { ... colors, logos ... },
    "payment_methods": [ {...}, {...} ],
  }
}
```

```

    "scale": { "generic": {...}, "prefix_map": [...] },
    "warehouse": { ... },
    "ksa_zatca": { ... }
  }
}

```

The cashier caches this in `frappe.cache()` keyed by terminal, invalidated by: -
`invalidate_boot_cache(terminal)` — manual call - Document hooks on Outlet, Profile, Theme

Other boot endpoints

Endpoint	Purpose
<code>pos_boot</code>	Full bootinfo for a terminal
<code>list_terminals_for_picker</code>	Terminal list for the picker UI
<code>get_default_terminal_for_session</code>	Currently returns <code>can_change</code> only (deprecated default-terminal path)
<code>invalidate_boot_cache(terminal)</code>	Clear cache for a terminal

Manager PIN security module

✓ **Working** (v15.5.13)

Module: `alphax_pos_suite.alphax_pos_suite.security.manager_pin`

Public API

```

@frappe.whitelist()
def set_manager_pin(user, pin) -> dict
    # System Manager only. Hashes PIN, upserts, resets lockout.

@frappe.whitelist()
@frappe.rate_limit(limit=10, seconds=300)
def verify_manager_pin(user, pin, action_type, terminal, outlet) -> dict
    # The hot path. Per-manager exponential lockout. Generic error
    # messages on any failure (no leakage). Rate-limited.

@frappe.whitelist()
def reset_manager_lockout(user) -> dict
    # System Manager only. Manual unlock.

@frappe.whitelist()
def log_action(action_type, terminal, outlet, notes) -> dict
    # Audit log entry for follow-up actions after a PIN verify.

def reset_daily_counters() -> None
    # Scheduled. Resets lockout_count_today on all PIN records.

```

Lockout policy (v15.5.13)

Exponential backoff schedule:

```
LOCKOUT_SCHEDULE_MINUTES = [  
    5,          # Lockout #1  
    30,         # Lockout #2  
    4 * 60,     # Lockout #3  
    24 * 60,    # Lockout #4 + email alert to System Manager  
]  
# Past index 3: admin reset required (locked_until set 10 years out)
```

Audit log model

The AlphaX POS Manager Authorization Log doctype is **append-only**: - Permissions: read for System Manager + AlphaX POS Manager, no write/create/delete for anyone - Entries created only via `frappe.get_doc(...).insert(ignore_permissions=True)` in `_audit_log()` helper

This means a compromised manager account cannot scrub their own audit trail through the Frappe UI.

Threat model

In scope: - Brute-force PIN attempts → exponential lockout + IP rate limit - Audit trail tampering → append-only doctype, ignore_permissions only - PIN database leak → bcrypt 12-round hashing - User enumeration via timing → bcrypt dummy hash on user-not-found path

Out of scope (deferred): - Phishing the PIN out of a manager → user education - Compromised cashier PC with keylogger → endpoint security - Insider with system manager role → no defense (by design; ultimate trust)

Domain pack system

✅ **Working** for the data model 🧪 **Beta** for cashier-side feature toggling (some packs not fully wired)

How it works

AlphaX POS Domain Pack doctype defines feature flags. Each AlphaX POS Outlet links to one domain pack.

```
# AlphaX POS Domain Pack  
{  
    "name": "Restaurant",  
    "show_floor_plan": 1,  
    "show_kds": 1,  
    "show_modifiers": 1,  
    "auto_print_kitchen_ticket": 1,  
    "service_charge_enabled": 1,
```

```

    "tips_enabled": 1,
    "table_management_enabled": 1,
    "show_pharmacy_features": 0,
    "show_serial_capture": 0,
    ...
}

```

Cashier consumption

The cashier reads the domain pack via pos_boot response:

```

// Vue cashier
const useStore = defineStore('settings', () => {
  const showFloorPlan = computed(() =>
boot.value?.domain_pack?.show_floor_plan)
  return { showFloorPlan }
})

// Classic cashier
if (state.boot?.domain_pack?.show_floor_plan) {
  $('[data-area="floor-plan-button"]').show()
}

```

Adding a new pack

1. Create a record in AlphaX POS Domain Pack
2. Set the relevant feature flags
3. Link from outlets

To add new feature flags, add Check fields to the Domain Pack doctype JSON, then teach the cashier UI to read them.

Soft adapter pattern (ZATCA, hardware bridge)

The “soft adapter” pattern lets us integrate with optional dependencies without breaking when they’re absent.

Example: ZATCA adapter

alphax_pos_suite/integrations/zatca_adapter.py:

```

def on_pos_invoice_submit(invoice, method):
    """Sales Invoice on_submit hook for ZATCA."""
    try:
        from alphax_zatca.api import submit_invoice
        result = submit_invoice(invoice)
        # log result to AlphaX POS Processing Log
    except ImportError:
        # alphax_zatca not installed; this is fine for non-KSA shops
        return

```

```
except Exception as e:
    # ZATCA failed; log but don't block the sale
    frappe.log_error(...)
```

The same module also tries the upstream `zatca_erp_gulf` if our fork isn't available — graceful fallback.

Example: Bridge adapter

The browser tries `ws://localhost:9876` for the bridge daemon. On WebSocket open failure, it silently falls back to browser-native print. The cashier never blocks on hardware availability.

Why this matters

A customer can install only `alphax_pos_suite` and have a working POS. ZATCA, the bridge, fancy printers — all optional. **The base case always works.**

Custom field strategy: no migrations

✓ Working

We never write database migrations to add fields. Instead:

1. Custom fields are declared in `data/custom_fields_seed.json`
2. `install.py::create_custom_fields()` reads the JSON and creates them via `frappe.create_custom_field(...)` (idempotent)
3. `after_install` and `after_migrate` both run this — so upgrades pick up new fields automatically

```
// data/custom_fields_seed.json
[
  {
    "doctype": "Custom Field",
    "dt": "Sales Invoice",
    "fieldname": "alphax_outlet",
    "label": "AlphaX POS Outlet",
    "fieldtype": "Link",
    "options": "AlphaX POS Outlet",
    "insert_after": "is_pos"
  },
  ...
]
```

To add a custom field

1. Add an entry to the JSON
2. Don't write a migration — `create_custom_fields()` will install it
3. Test on a fresh site AND an existing site

To deprecate a custom field

1. Mark it deprecated in the JSON description
 2. Don't delete it (deletion requires a real migration)
 3. Future cleanup: write a delete-custom-field patch in patches/v15_0/
-

Extending the system

Adding a new doctype

1. Use Frappe's standard bench new-ddoctype workflow OR create the JSON manually
2. Place it in `alphax_pos_suite/alphax_pos_suite/doctype/<your_ddoctype>/`
3. Set module: "AlphaX POS Suite" in the JSON
4. (Optional) Add a controller .py file
5. Run `bench --site ... migrate`

Adding a new payment method

1. Standard ERPNext: create a Mode of Payment record
2. Link from POS Profile → Mode of Payment table
3. (Optional) For card terminals: add a record in AlphaX POS Card Transaction when capturing card response data

Adding a new domain pack

See "Domain pack system" above.

Adding a new cashier action

In the Classic cashier (`alphax_pos_classic.js`):

```
// In the HTML template
<button class="alphax-btn" data-action="my_new_action">My
Action</button>

// Handler
$(wrapper).on('click', '[data-action="my_new_action"]', function(){
  // Your code here
});
```

In the Vue cashier (`alphax_pos_v2/sfc/components/...`):

```
<template>
  <button @click="onMyNewAction">My Action</button>
</template>
<script setup>
function onMyNewAction() {
  // Your code here
}
</script>
```

Adding a manager-authorized action

```
// 1. Show the manager PIN dialog (function exists in
alphax_pos_classic.js)
open_manager_dialog();

// After successful verify, the dialog opens its post-auth menu.
// Add a button there that triggers your action.

// 2. From your action handler, call the audit log endpoint
await frappe.call({
  method:
'alphax_pos_suite.alphax_pos_suite.security.manager_pin.log_action',
  args: { action_type: 'My Custom Action', terminal: pc_terminal() }
});

// 3. Then perform the action
```

Adding a hook

Edit `hooks.py`. Don't add hooks at runtime — they need to be in the hook file for Frappe's hook resolver to find them.

Customer onboarding playbook

When a new customer signs up:

Phase 1: discovery (1-2 days)

- ☐ Get their company info, VAT, branches, hardware list
- ☐ Confirm Frappe Cloud bench is provisioned
- ☐ Get GitHub access
- ☐ Walk through the Administrator Manual together (skim, not deep)
- ☐ Set realistic timeline expectations: 1-2 weeks for non-trivial setups; 3-4 weeks if pharmacy or restaurant features are needed

Phase 2: setup (3-5 days)

- ☐ Day 1-2: Deploy apps + create company hierarchy
- ☐ Day 3: Items, taxes, pricing
- ☐ Day 4: Users, roles, manager PINs
- ☐ Day 5: First test cashier session end-to-end

Phase 3: ZATCA + hardware (3-5 days)

- ☐ Sandbox ZATCA setup, test invoice submissions
- ☐ Install bridge daemons on each cashier PC (if needed)
- ☐ Test printers, drawers, scanners
- ☐ Switch ZATCA to production

Phase 4: training (2-3 days)

- ☐ Train managers using Manager Manual
- ☐ Each manager sets up their PINs, learns the workflow
- ☐ Train cashiers using Cashier Manual
- ☐ Run a full simulated business day with all staff

Phase 5: go-live (1 day)

- ☐ Final sanity check
- ☐ Switch from test invoices to real ones
- ☐ On-call support for the first 24 hours

Phase 6: handoff (ongoing)

- ☐ Show them the Administrator Manual sections they'll use weekly
 - ☐ Set up support escalation: customer → their IT → you
 - ☐ Schedule a 30-day check-in
-

Known issues and gotchas

Frappe Cloud asset symlink quirk

Documented above (Section 4). Workaround: keep all assets under `dist/vendor/`.

Repo name vs app name

Repo is `alphax-pos-core-v15.5.1`, app is `alphax_pos_suite`. Mostly fine, but causes occasional bench-tooling confusion. Plan to rename the repo eventually.

Long-conversation context loss in our AI dev cycles

The previous AI session that wrote much of this code occasionally needed re-grounding on what was already done. Symptom: ships a “fix” that re-introduces a bug fixed two versions ago.

Mitigation for human developers: when modifying versioned code, look at the git history before re-doing fixes.

Vue cashier vendor bundles

Sometimes the cashier loads vendor bundles from CDN even when local copies exist. Cause: timing race between local fetch and CDN fallback. Acceptable for now; the user gets a slightly slower first load.

Single-PIN-per-manager limit

Each manager has at most one PIN. Cannot have separate PINs for “floor-supervisor PIN” vs “back-office PIN”. Acceptable for v15.5.13; revisit in v16 if customers ask.

No multi-factor auth on the PIN

A 6-digit PIN is 1M combinations. Combined with our exponential lockout, this is secure for normal threat models. For high-value shops, consider: - Restricting manager-PIN-able actions to specific terminals - Requiring two-of-N approvals for high-value voids - Adding TOTP as a second factor on top of the PIN

These are deferred features.

Roadmap and deferred work

Likely v15.6

- Vue cashier reaches production parity with Classic
- Inline prescription capture in cashier
- Floor plan integrated with cashier (table-tap → cart)
- KDS production-ready
- Manager PIN admin UI (currently CLI-only)

Likely v16

- Full ESLint/Prettier on Classic JS
- Replace runtime SFC compilation with Vite build (if Frappe Cloud supports it cleanly)
- Multi-PIN-per-manager (different actions, different PINs)
- TOTP second factor
- Per-branch ACLs on manager authorization

Wishlist

- ZATCA Phase 3 readiness
 - Hardware bridge production hardening
 - Code-signing certificates for installer (Windows EV, Apple Dev)
 - Native mobile app (currently runs in mobile browser, but native could be smoother)
 - Restaurant-specific:
 - Reservation system
 - Table-side ordering tablet UI
 - Pharmacy-specific:
 - NPHIES integration (KSA insurance)
 - Inventory expiry alerts
 - Multi-language UI (Arabic + English fully)
-

Appendix A: quick reference

Critical paths

- Workspace: /app/alphax-pos-hub

- Classic cashier: /app/alphax-pos-classic
- Vue cashier: /app/alphax-pos-v2
- Settings: /app/alphax-pos-settings/AlphaX POS Settings
- Audit log: /app/alphax-pos-manager-authorization-log
- ZATCA Status: /app/alphax-pos-processing-log

Critical files

File	What's in it
alphax_pos_suite/hooks.py	All hook registrations
alphax_pos_suite/ alphax_pos_suite/install.py	after_install chain
alphax_pos_suite/ alphax_pos_suite/boot/api.py	pos_boot endpoint
alphax_pos_suite/ alphax_pos_suite/security/ manager_pin.py	PIN auth
alphax_pos_suite/ alphax_pos_suite/page/ alphax_pos_classic/ alphax_pos_classic.js	Production cashier
alphax_pos_suite/fixtures/ workspace.json	Workspace structure
build_workspace.py	Workspace generator script
data/custom_fields_seed.json	All custom fields

Critical commands

Deploy a new version

`git push` *# triggers Frappe Cloud auto-build*

Then on Frappe Cloud: Bench → Show Updates → Deploy

Set a manager PIN (no UI yet)

`bench --site <site> console`

`>>> import frappe;`

`frappe.call('alphax_pos_suite...security.manager_pin.set_manager_pin',
user='x@y.com', pin='1234')`

Reset a lockout

`>>>`

`frappe.call('alphax_pos_suite...security.manager_pin.reset_manager_loc
kout', user='x@y.com')`

Force rebuild assets

`bench --site <site> build --app alphax_pos_suite --hard-link`

Migrate (run all patches)

`bench --site <site> migrate`

```
# Run scheduler manually (for testing daily resets)
bench --site <site> execute
alphax_pos_suite.alphax_pos_suite.security.manager_pin.reset_daily_counters
```

Manual version 15.5.13 — May 2026. Maintained alongside the source repository.